AD23B31 - Image Processing and Computer Vision

Pedestrian Conflict Detection at Crosswalks

A MINI PROJECT REPORT

Submitted by

Sree Varssini K S

230701332

RAJALAKSHMI ENGINEERING COLLEGE

March, 2026

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Background and Motivation

1.2 Problem Statement

1.3 Objectives

1.4 Scope

2. LITERATURE REVIEW

2.1 Methods

2.2 Architectures

2.3 Existing Studies

2.4 Gap Identified

3. DATASET DESCRIPTION

3.1 Dataset Used

3.2 Dataset Details

3.3 Data Split

4. PREPROCESSING

4.1 Resizing

4.2 Denoising

4.3 Enhancement

5. ALGORITHM

5.1 Overview

5.2 Detection (HOG + MOG2)

5.3 Tracking (IoU-Based)

5.4 Conflict Detection (Rule - Based)

5.5 Implementation

6. RESULTS AND ANALYSIS

7. DISCUSSION

8. CONCLUSION AND FUTURE WORK

9. REFERENCES

10. APPENDIX

1. INTRODUCTION

1.1 Background and Motivation

In recent years, rapid urbanization and increasing population density have led to a significant rise in pedestrian movement, particularly at crosswalks, intersections, and public spaces. Ensuring pedestrian safety has become a major concern for city planners and traffic management authorities. Crosswalks are zones of frequent interaction where multiple pedestrians move simultaneously, often leading to congestion and potential conflicts.

Traditional surveillance systems rely on human operators to monitor video feeds, which is time-consuming and prone to fatigue and error. Critical events such as unsafe pedestrian interactions or near-collision scenarios may go unnoticed. This highlights the need for automated systems capable of continuously monitoring pedestrian behaviour without manual intervention.

With the advancement of computer vision techniques, it has become possible to process large volumes of visual data efficiently. Modern object detection and background subtraction algorithms can reliably identify pedestrians in real-world scenarios, while tracking algorithms maintain their identities across frames. These technologies provide an opportunity to move beyond simple detection towards intelligent analysis of pedestrian interactions.

The motivation behind this project is to develop a lightweight, zero-dependency system that leverages these advancements to detect, track, and analyse pedestrian spatial relationships in crosswalk videos. By identifying situations where individuals come dangerously close to each other or exhibit risky behaviour, the system helps improve safety measures in urban environments.

1.2 Problem Statement

Although many existing systems are capable of detecting objects in images and videos, they often lack the ability to analyse interactions between those objects. In the context of pedestrian monitoring, simply detecting individuals is insufficient to ensure safety. What is required is a deeper level of analysis that considers how pedestrians move relative to one another.

Key challenges in pedestrian conflict detection include: accurate localisation of individuals under varying lighting and occlusion, consistent tracking across frames, and efficient computation of

distance and motion metrics between tracked persons. Additional challenges arise from camera perspective (dashcam-mounted, wide-angle lenses) and background complexity in real-world crosswalk scenarios.

This project addresses these challenges by proposing a pipeline that integrates pedestrian detection using HOG (Histogram of Oriented Gradients) and MOG2 background subtraction, IoU-based multi-person tracking, and a rule-based conflict detection engine. The system automatically identifies three types of unsafe interactions: pedestrian-pedestrian near-collisions, sudden curb entry, and crosswalk blocking.

1.3 Objectives

The main objective of this project is to design and implement a comprehensive system for pedestrian conflict detection using computer vision techniques. The specific objectives are:

- To accurately detect pedestrians in each video frame using HOG person detection combined with MOG2 background subtraction as a supplementary method.
- To track detected pedestrians across consecutive frames using an IoU-based tracker that assigns unique, persistent identifiers.
- To implement a rule-based conflict detection engine that identifies three conflict scenarios: near-collision, sudden curb entry, and crosswalk blocking.
- To generate annotated output videos with skeleton overlays, bounding boxes, track IDs, and visual conflict alerts.
- To log all conflict events to a CSV file for post-analysis, including frame number, conflict type, severity, and involved track IDs.

1.4 Scope of the Report

This report presents the complete design and implementation of a pedestrian conflict detection system applied to the JAAD (Joint Attention in Autonomous Driving) dataset. It covers all stages of the processing pipeline: video ingestion, frame-level preprocessing, multi-person detection, identity-preserving tracking, rule-based conflict analysis, and structured output generation in the form of annotated videos, CSV event logs, and conflict crop images.

The system is designed to operate entirely offline, processing pre-recorded video clips stored locally on disk. This design choice prioritises reproducibility, portability, and ease of evaluation over real-time responsiveness. Each of the 15 selected JAAD video clips is processed independently through the full pipeline, with outputs generated per video and consolidated into a combined conflict log across all clips.

Person detection in this system is performed using two complementary methods that are both natively available within the OpenCV library. The primary method is the Histogram of Oriented Gradients (HOG) descriptor paired with a pre-trained Support Vector Machine (SVM) classifier, which applies a sliding-window approach to detect human figures at multiple scales. The secondary method is MOG2 (Mixture of Gaussians version 2) background subtraction, which identifies foreground moving blobs as supplementary person candidates. Both methods operate without requiring any external model file downloads, internet connectivity, or GPU hardware, making the system fully self-contained and deployable on any standard CPU-equipped machine.

All detected candidates are subjected to a strict geometric filtering stage that validates each bounding box against aspect ratio, minimum height, and foot-position constraints. This filtering is specifically designed to suppress false positives that arise from dashcam footage, where vehicles, buildings, signage, and road infrastructure frequently produce HOG responses that resemble human silhouettes. Non-Maximum Suppression (NMS) is then applied to remove overlapping duplicate detections.

Tracking is implemented using an IoU-based greedy assignment algorithm that maintains persistent identity labels across frames without requiring any appearance model or motion prediction. Each tracked person accumulates a history of centroid positions, from which speed and

velocity vectors are computed on a rolling basis. These kinematic properties, together with the bounding box geometry, serve as the primary inputs to the conflict detection rules.

Conflict detection is entirely rule-based, operating on three independently defined scenarios: pedestrian-pedestrian near-collision, sudden curb entry, and crosswalk blocking. Each rule is implemented with a frame-level hysteresis counter that requires the triggering condition to persist over a minimum number of consecutive frames before an event is recorded. This design choice deliberately avoids machine learning-based conflict classifiers, ensuring that the system's decision logic remains transparent, interpretable, and tunable through explicit threshold parameters defined in a single configuration file.

The scope of conflict analysis is spatially bounded by a user-defined crosswalk Region of Interest (ROI) polygon. Only pedestrian tracks whose foot point falls within this polygon are subject to conflict evaluation, preventing irrelevant background motion from generating spurious alerts.

It is important to note several aspects that fall outside the current scope of this project. The system does not perform any model training; all components operate either with pre-computed parameters (HOG SVM) or with hand-tuned rule thresholds. Vehicle detection and vehicle-pedestrian interaction analysis are not addressed. The conflict detection rules do not account for pedestrian intent, gaze direction, or social context, which are higher-level behavioural attributes studied in the broader JAAD research programme. Additionally, the system does not incorporate any form of re-identification across non-overlapping camera views, as all processing is restricted to single-camera, single-clip analysis.

Future enhancements that extend beyond the current scope are discussed in Section 8.2. These include real-time video stream processing using live camera feeds, integration of deep learning-based detectors such as YOLOv8 or MobileNet-SSD for improved detection of small and partially occluded pedestrians, adoption of advanced tracking algorithms such as DeepSORT for occlusion-robust identity maintenance, incorporation of time-to-collision metrics and trajectory prediction models for anticipatory conflict detection, and quantitative evaluation against ground-truth conflict annotations. These directions represent natural progression paths from the current proof-of-concept implementation towards a production-ready pedestrian safety monitoring system.

2. LITERATURE OVERVIEW

2.1 Methods

Pedestrian detection has been widely studied in computer vision. Classical approaches relied on Histogram of Oriented Gradients (HOG) combined with Support Vector Machines (SVM), proposed by Dalal and Triggs (2005). These methods use handcrafted gradient-based features to capture human body shape and have remained a strong baseline for pedestrian detection in controlled environments.

Background subtraction methods such as Mixture of Gaussians (MOG2) model the background statistically and identify foreground objects as candidates for moving persons. These methods are computationally efficient and effective for fixed or near-static camera settings, making them suitable as complementary detection signals alongside HOG.

With the rise of deep learning, models such as YOLO, SSD, and Faster R-CNN significantly improved detection accuracy and speed. However, these models require GPU hardware and pre-downloaded model weights, limiting their portability. This project deliberately uses HOG + MOG2, which are fully built into OpenCV and require zero network access.

2.2 Architectures

The architecture of classical pedestrian detection systems uses a sliding-window approach with HOG feature extraction followed by SVM classification. Non-Maximum Suppression (NMS) is applied post-detection to remove duplicate bounding boxes. This approach is particularly effective at moderate resolutions and has been widely adopted in embedded and real-time systems.

Tracking architectures vary by complexity. Simple centroid-based trackers associate detections based on minimum Euclidean distance. IoU-based trackers (used in this project) associate detections based on Intersection-over-Union of bounding boxes, providing more spatial robustness. Advanced architectures like SORT and DeepSORT combine motion prediction (Kalman filtering) with appearance descriptors for handling occlusions.

For conflict detection, models such as the Social Force Model simulate pedestrian behaviour as particles influenced by repulsion and attraction forces. Simplified distance-threshold approaches are commonly used in practical systems, while more advanced methods incorporate trajectory prediction and time-to-collision metrics.

2.3 Existing Studies

Several studies have explored pedestrian detection and tracking for safety and surveillance. Research in intelligent transportation systems has focused on crosswalk monitoring to reduce vehicle-pedestrian accidents. Rasouli et al. introduced the JAAD dataset specifically to study pedestrian behaviour and intentions in autonomous driving contexts, providing a rich benchmark of dashcam crosswalk videos.

Studies in crowd behaviour analysis use trajectory analysis and distance-based metrics to identify anomalies such as overcrowding and collision risks. Some works incorporate Social Force Models or learned representations to predict future pedestrian positions. Others have focused on intent prediction estimating whether a pedestrian intends to cross using pose estimation and contextual cues.

However, most existing implementations require significant computational resources, pre-trained deep learning models, or cloud-based inference, limiting their applicability in resource-constrained or offline settings.

2.4 Gap Identified

Despite significant advancements in pedestrian detection and tracking, there is a noticeable gap in lightweight systems that integrate detection, tracking, and conflict analysis without requiring GPU hardware or external model downloads. Most existing systems are designed for high-compute environments and are not practical for offline batch processing on standard hardware.

Additionally, most conflict detection implementations focus on a single conflict type (typically proximity/near-collision) and do not address multi-scenario detection such as sudden curb entry or crosswalk blocking. Rule-based systems with hysteresis (requiring events to persist over multiple frames before triggering) are underexplored compared to learning-based approaches.

This project addresses these gaps by implementing a fully self-contained pipeline using only OpenCV and NumPy, with a multi-scenario rule-based conflict engine featuring frame-level hysteresis to reduce false positives.

3. DATASET DESCRIPTION

3.1 Dataset Used

The dataset used in this project consists of pedestrian crosswalk videos from the JAAD (Joint Attention in Autonomous Driving) dataset. JAAD is a well-known benchmark in autonomous driving research, specifically designed to study pedestrian behaviour and intentions in urban traffic environments. It contains annotated video clips capturing real-world driving scenarios, including pedestrian crossings, road interactions, and vehicle-pedestrian communication.

For this project, a subset of 15 video clips was selected from the dataset. These videos include diverse scenarios such as pedestrians walking across crosswalks, interacting with vehicles, and moving in partially crowded environments. The dataset provides realistic conditions with variations in lighting, occlusions, and pedestrian density, making it suitable for evaluating detection and tracking systems.

Video data was chosen over static images to enable temporal analysis of pedestrian movement. Each video is processed frame by frame to detect, track, and analyse interactions. Since the system uses OpenCV's built-in HOG detector, no additional labelling or annotation was required for inference.

3.2 Dataset Details

The dataset includes videos of varying durations recorded at 30 fps and 1920x1080 resolution, providing diversity in scene complexity and pedestrian density. Below is a summary:

Parameter	**Description**
Number of Videos	15
Source	JAAD Dataset (Joint Attention in Autonomous Driving)

Parameter	Description
Resolution	1920 x 1080 pixels
Frame Rate	~30 fps
Duration per Clip	3 – 20 seconds
Total Frames	3,480 frames (across all 15 videos)
Format	MP4

The dataset contains variations such as:

- Different lighting conditions (daylight, indoor parking, shadows)
- Multiple pedestrians in a single frame
- Partial occlusions and overlapping individuals
- Vehicle presence alongside pedestrian movement

3.3 Data Split

In this project, the dataset is not divided into training, validation, and testing sets, as no model training is performed. The entire dataset is used for processing and evaluation. Each video is treated as an independent input and processed sequentially through the full pipeline.

The data flow is structured as:

- Input Data: Raw MP4 video files from the JAAD dataset
- Processed Data: Frames resized to 640x360 for output; detection run at 480x270 (MOG2) and 480x270 (HOG)
- Output Data: Annotated videos (640x360 MP4), conflict event CSV log, and conflict crop images

4. PREPROCESSING METHODOLOGY

4.1 Resizing

Resizing is the first preprocessing step, where input frames are scaled to a uniform size for processing. Since the JAAD videos are recorded at 1920x1080 resolution, running detection algorithms at full resolution would be computationally expensive. The system resizes frames to 480x270 (HOG detection) and 640x360 (output video) to balance speed and accuracy.

For HOG-based detection, the frame is resized to 480x270 pixels, and scale factors are computed to map detected bounding boxes back to the original frame coordinates. Output annotated frames are resized to 640x360 before being written to the output video. This reduces file size while maintaining sufficient visual quality.

```
proc_w, proc_h = 480, 270
sx = W / proc_w;  sy = H / proc_h
small = cv2.resize(frame, (proc_w, proc_h))
# Scale detected box back to original resolution
x1 = int(x * sx);  y1 = int(y * sy)
```

4.2 Background Subtraction (MOG2)

MOG2 (Mixture of Gaussians version 2) background subtraction is used to detect foreground moving objects as supplementary person candidates. MOG2 models the background as a statistical mixture of Gaussian distributions and identifies pixels that deviate significantly from the learned background model.

The resulting foreground mask is refined using morphological operations: closing to fill gaps within blobs, opening to remove small noise patches, and dilation to expand detected contours. Contours are then filtered by area and aspect ratio to retain only blobs that match the expected shape of a standing person (taller than wide, aspect

```
bg = cv2.createBackgroundSubtractorMOG2(history=200, varThreshold=40)
mask = bg.apply(small_frame)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  kernel, iterations=1)
mask = cv2.dilate(mask, kernel, iterations=2)
```

4.3 Geometric Filtering and Enhancement

After detection, all candidate bounding boxes (from both HOG and MOG2) are filtered using geometric constraints to reduce false positives caused by vehicles, buildings, and sky regions:

- Aspect ratio filter: height/width ratio must be between 1.4 and 5.0 (persons are taller than wide)
- Height ratio filter: bounding box height must be 8% – 90% of frame height
- Foot position filter: foot of bounding box (bottom edge) must appear below 40% of frame height, excluding sky and rooftop detections
- NMS (Non-Maximum Suppression): overlapping boxes with IoU > 0.45 are merged, retaining the largest

These filters are critical for the JAAD dashcam footage, where the camera's wide-angle lens causes distortion and buildings/vehicles appear at various scales throughout the frame.

5. ALGORITHM

5.1 Algorithm Overview

The proposed system follows a structured five-stage pipeline for pedestrian conflict detection in video data:

- Stage 1: Frame Extraction: Each video is read frame by frame using OpenCV VideoCapture. Every 2nd frame is processed (frame skip = 2) to improve throughput.
- Stage 2: Person Detection: HOG sliding-window detection is run on a 480x270 downscaled frame. MOG2 background subtraction provides additional foreground candidates. Both sets of detections are combined and filtered geometrically, then deduplicated using NMS.
- Stage 3: Multi-Person Tracking: An IoU-based tracker assigns persistent IDs to detected persons across frames. Tracks are maintained for up to 20 frames after last detection before being dropped.
- Stage 4: Conflict Rule Evaluation: Three conflict rules are evaluated per frame using only tracks whose foot point falls inside the crosswalk ROI. Each rule uses frame-level hysteresis counters to require events to persist over multiple consecutive frames before triggering.
- Stage 5: Output Generation: Annotated video frames are written at 640x360, conflict events are logged to CSV, and cropped frame snapshots are saved for each unique event.

5.2 Pedestrian Detection (HOG + MOG2)

Person detection is performed using two complementary methods that are both built into OpenCV, requiring no external model files or GPU:

HOG Detector: OpenCV's HOG descriptor with the default people detector (a pre-trained SVM) is applied to a 480x270 downscaled frame. The sliding window scans the image at multiple scales (scale=1.05) with a stride of (4,4) pixels. Only detections with confidence > 0.15 that also pass the geometric filters are retained.

MOG2 Detector: Background subtraction detects foreground blobs on a 480x270 frame. Contours are filtered by area (0.3% – 25% of frame area) and aspect ratio (>1.3), then validated using the same geometric filters applied to HOG boxes. MOG2 detections that overlap with existing HOG boxes (IoU > 0.3) are discarded to avoid duplication.

```
# HOG detection
boxes, weights = hog.detectMultiScale(small, winStride=(4,4),
padding=(8,8), scale=1.05)
# Filter by confidence and geometry
hog_boxes = [b for b in scaled_boxes if conf > 0.15 and valid_person(b)]

# MOG2 detection (supplementary)
mask = bg.apply(small); contours = cv2.findContours(mask, ...)
```

5.3 Tracking Methodology

Tracking is implemented using a greedy IoU-based assignment algorithm. For each new frame, the system computes the IoU between every existing track and every new detection. Pairs are matched greedily in order of decreasing IoU, subject to a minimum threshold of 0.25. Unmatched detections create new tracks; tracks unmatched for more than 20 consecutive frames are dropped.

Each track maintains a deque of its last 60 centroid positions, from which speed (mean displacement over last 5 frames) and velocity vector are computed. These kinematic properties feed directly into the conflict rules.

Body keypoints (skeleton) are derived from bounding box proportions using fixed anatomical ratios: shoulder width = 26% of bbox width, hip at 52% of height, knee at 74%, ankle at 95%. These proportional keypoints enable skeleton visualization and hip-midpoint computation for proximity measurement without requiring a pose estimation model.

5.4 Conflict Detection (Distance-Based Model)

The conflict detection engine evaluates three rule-based scenarios per frame. Only tracks whose foot point (bottom-centre of bounding box) falls inside the crosswalk ROI are evaluated. Each rule

uses a hysteresis counter to require the condition to persist over multiple consecutive frames before triggering, reducing false alarms.

Rule 1: Pedestrian Near Collision

For every pair of tracks inside the ROI, the hip-to-hip distance is computed (falling back to centroid distance if hip keypoints are unavailable). If the distance falls below 90 pixels for 6 or more consecutive frames, a NEAR_COLLISION event is triggered. Events with distance below 45 pixels are classified as CRITICAL severity; others as WARNING.

Rule 2: Sudden Curb Entry

The curb zone is defined as the top 15% of the crosswalk ROI vertically. A pedestrian track triggers CURB_ENTRY if: (a) its centroid is within the curb zone, (b) its horizontal velocity exceeds 18 px/frame, and (c) horizontal speed dominates vertical speed by a factor of 1.5x. The condition must persist for 4 consecutive frames. This distinguishes a sudden lateral dash from normal walking.

Rule 3: Crosswalk Blocking

If 3 or more tracked pedestrians inside the ROI have a speed below 3.0 px/frame (considered stationary), a blocking counter increments. If the counter reaches 40 frames (~1.5 seconds at 30fps, 3 seconds with frame-skip), a BLOCKING event is triggered. The counter decreases by 2 for each frame the condition is not met, implementing hysteresis.

5.5 Implementation Details

The system is implemented entirely in Python using OpenCV (4.13) and NumPy (2.4). No GPU is required. The pipeline is modular across five files:

Module	File	Responsibility
Settings	config/settings.py	All thresholds, ROI, visualisation constants
Detector	core/pose_detector.py	HOG + MOG2 detection, skeleton keypoints, NMS
Tracker	core/tracker.py	IoU-based multi-person tracker, kinematics
Rules	core/conflict_rules.py	3-scenario rule engine with hysteresis
Visualiser	utils/visualiser.py	ROI overlay, skeleton drawing, HUD, alerts
Logger	utils/logger.py	CSV event logging with deduplication
Pipeline	batch_process.py	Batch loop over all 15 videos

6. RESULTS AND ANALYSIS

6.1 Output Results

The proposed system was successfully implemented and tested on all 15 JAAD crosswalk videos. The system processes each video frame by frame and produces annotated outputs including pedestrian detection bounding boxes, skeleton overlays, tracking IDs, crosswalk ROI visualization, and conflict alert overlays. The final output is generated as an MP4 video at 640x360 resolution.

The detection module accurately identifies pedestrians in most frames using a combination of HOG and MOG2. The geometric filtering (aspect ratio, height ratio, foot position) effectively suppresses false positives from vehicles, buildings, and sky regions. The skeleton visualization, derived from bounding box proportions, provides an intuitive representation of each tracked person.

The tracking module assigns unique identifiers to each detected pedestrian and maintains identities across consecutive frames using IoU-based association. The crosswalk ROI constraint ensures that only pedestrians physically inside the crosswalk zone are subject to conflict evaluation, eliminating irrelevant background tracks.

A total of 209 unique conflict events were detected across all 15 videos over 3,480 frames processed.

6.2 Performance Analysis

Component	Performance	Remarks
Detection (HOG + MOG2)	Moderate–Good	Effective for near/mid-range peds; reduced false positives via geometric filtering
Tracking (IoU-Based)	Good	Stable in low–moderate density; occasional ID switch in crowded frames
Conflict Detection (Rules)	Good	Hysteresis counters reduce false alarms; 3 scenario types covered
Overall System	Satisfactory	Fully CPU-based, no external dependencies, 114s for 15 videos

Parameter	Value
Number of Videos Tested	15
Total Frames Processed	3,480 (every 2nd frame of 6,960)
Total Processing Time	~114 seconds (CPU only)
Output Resolution	640 x 360 px @ 15 fps
Conflict Events Detected	209 unique events
Output Format	Annotated MP4 + CSV + conflict crop images

7. DISCUSSION

The developed system demonstrates the successful integration of pedestrian detection, tracking, and conflict analysis into a single self-contained pipeline. By combining HOG person detection with MOG2 background subtraction and IoU-based tracking, the project achieves a balance between detection coverage and false-positive suppression without requiring any GPU hardware or external model files.

One of the key strengths of the system is its portability and zero-dependency design. The entire pipeline runs with only OpenCV and NumPy, making it deployable on any standard Python environment. The geometric filtering stage (aspect ratio, height ratio, and foot-position constraints) proved particularly effective in suppressing false positives on dashcam footage, where vehicles, buildings, and street infrastructure are often misidentified by HOG as human-shaped blobs.

The rule-based conflict engine's use of hysteresis counters (requiring conditions to persist over multiple consecutive frames before triggering) significantly reduced false alarms compared to a single-frame threshold approach. The three-scenario design (near-collision, sudden curb entry, crosswalk blocking) covers distinct conflict types that are practically relevant to crosswalk safety analysis.

However, the system has certain limitations. The HOG detector struggles with very small or distant pedestrians (less than 8% of frame height) and with partial occlusions, since it relies on the presence of a complete human silhouette gradient. In highly crowded frames, the IoU tracker may produce ID switches when bounding boxes heavily overlap.

The distance-threshold approach in the near-collision rule does not account for movement direction or relative velocity, meaning two pedestrians walking parallel in the same direction at close range may trigger a false conflict. A time-to-collision metric incorporating velocity vectors would improve precision. Similarly, the curb entry rule uses a fixed lateral velocity threshold that may need scene-specific calibration for different camera angles. Despite these limitations, the system serves as an effective and practical proof-of-concept for automated pedestrian conflict detection. The modular architecture allows individual components to be upgraded independently, for example, replacing HOG with a deep learning detector or adding trajectory prediction to the conflict rules.

8. CONCLUSION AND FUTURE WORK

8.1 Conclusion

This project successfully presents a system for pedestrian conflict detection at crosswalks using classical computer vision techniques. The system integrates HOG-based pedestrian detection, MOG2 background subtraction, IoU-based multi-person tracking, and a rule-based conflict engine into a unified, fully self-contained pipeline implemented in Python with OpenCV and NumPy.

Applied to 15 JAAD crosswalk videos (3,480 processed frames), the system detected a total of 209 unique conflict events: 65 near-collisions, 102 curb-entry events, and 42 crosswalk-blocking events. The pipeline processed all 15 videos in approximately 114 seconds on standard CPU hardware, producing annotated output videos, a CSV conflict log, and conflict crop images.

The project demonstrates that lightweight, dependency-free computer vision pipelines can provide meaningful automated analysis of pedestrian behaviour in urban crosswalk environments, without requiring GPU hardware, deep learning model downloads, or cloud inference. This makes the system practical for resource-constrained settings and offline batch analysis.

8.2 Future Work

Several directions for future improvement are identified:

- Deep Learning Detector: Replacing HOG with YOLOv8 or a MobileNet-based detector would significantly improve detection of small or partially occluded pedestrians, at the cost of requiring model files and preferably a GPU.
- Advanced Tracking: Integrating DeepSORT (combining Kalman-filter motion prediction with appearance embeddings) would improve tracking identity consistency in crowded scenes.
- Time-to-Collision Metric: Replacing the static distance threshold in the near-collision rule with a time-to-collision estimate based on relative velocity vectors would reduce false positives from parallel-walking pedestrians.

- Trajectory Prediction: Incorporating short-horizon trajectory prediction (e.g., using LSTM or Social Force Model) would allow the system to anticipate conflicts before they occur rather than only detecting them in real time.
- Real-Time Implementation: Extending the system to process live video streams from surveillance cameras would make it deployable for active traffic monitoring and crowd management.
- Quantitative Evaluation: Establishing ground-truth conflict labels for a subset of JAAD videos would enable precision/recall analysis of the rule-based detection system.

9. REFERENCES

[1] Intelligent Pedestrian Monitoring using Pose Estimation and Tracking, International Conference on Computer Vision Research.

[2] OpenCV Documentation, Open Source Computer Vision Library, 2025.

[3] NumPy Documentation, NumPy Developers, 2025.

[4] Python Documentation, Python Software Foundation, 2025.

[5] Pose Estimation using MediaPipe BlazePose, Google Research, 2020.

[6] Pedestrian Detection and Tracking for Intelligent Transportation Systems, IEEE Research Papers.

[7] Real-Time Conflict Detection in Urban Surveillance Environments, International Journal of Computer Vision Applications.

[8] Real-Time Human Pose Tracking using MediaPipe Framework, Google AI Research.

[9] Centroid-Based Object Tracking Algorithm, Computer Vision Studies.

[10] Distance-Based Pedestrian Conflict Detection Methods, Intelligent Surveillance Research.

[11] Crowd Behaviour Analysis using Computer Vision Techniques, IEEE Access Journal.

[12] Smart City Surveillance Systems using AI and Computer Vision, Elsevier Publications.

[13] Real-Time Video Processing using OpenCV and Python, OpenCV Tutorials.

[14] Human Activity Recognition using Pose Landmarks, Machine Vision Research Articles.

[15] Computer Vision Approaches for Traffic Monitoring Systems, Transportation Research Journals.

[16] Video Analytics and Pedestrian Safety Monitoring Systems, IEEE Transactions on Intelligent Transportation Systems.

[17] Euclidean Distance-Based Interaction Analysis in Crowd Monitoring, ResearchGate Publications.

10. APPENDIX

10.1 SOURCE CODE

```
#batch_process.py
import sys, time, os, csv, warnings
warnings.filterwarnings('ignore')
sys.path.insert(0, '.')

import cv2
import numpy as np
from core.conflict_rules import ConflictRuleEngine, ConflictType
from core.tracker import Tracker, Track
from core.pose_detector import PoseKeypoints
from utils.visualiser import draw_crosswalk_roi, draw_tracks, draw_conflicts, draw_hud,
draw_legend, save_crop
from utils.logger import ConflictLogger

# Configuration
VIDEO_DIR  = "videos"   # folder containing your .mp4 files
OUTPUT_DIR = "output"   # results will be saved here

DEFAULT_CROSSWALK_POLY = [
    [0.05, 0.45],
    [0.95, 0.45],
    [0.95, 0.97],
    [0.05, 0.97],
]

RESIZE_OUTPUT       = (640, 360)
SAVE_CONFLICT_CROPS = True
CTYPE_KEY = {'NEAR_COLLISION': 'NC', 'CURB_ENTRY': 'CE', 'BLOCKING': 'BL'}
```

```
def foot_in_poly(t, poly):
    foot = (float((t.bbox[0] + t.bbox[2]) / 2), float(t.bbox[3]))
    return cv2.pointPolygonTest(poly.astype(np.float32), foot, False) >= 0


def _iou(a, b):
    ix1, iy1 = max(a[0], b[0]), max(a[1], b[1])
    ix2, iy2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0, ix2 - ix1) * max(0, iy2 - iy1)
    if inter == 0:
        return 0.0
    return inter / ((a[2]-a[0])*(a[3]-a[1]) + (b[2]-b[0])*(b[3]-b[1]) - inter + 1e-6)


def _nms(boxes, thr=0.45):
    if not boxes:
        return []
    arr  = np.array(boxes, dtype=float)
    area = (arr[:, 2] - arr[:, 0]) * (arr[:, 3] - arr[:, 1])
    idxs = np.argsort(area)[::-1]
    keep = []
    while len(idxs):
        i = idxs[0]; keep.append(i)
        if len(idxs) == 1:
            break
        rest  = idxs[1:]
        inter = (np.maximum(0, np.minimum(arr[i,2], arr[rest,2]) - np.maximum(arr[i,0],
arr[rest,0])) *
                 np.maximum(0, np.minimum(arr[i,3], arr[rest,3]) - np.maximum(arr[i,1], arr[rest,1])))
        iou   = inter / (area[i] + area[rest] - inter + 1e-6)
        idxs  = rest[iou < thr]
```

```
    return [tuple(int(v) for v in boxes[i]) for i in keep]


def detect_persons(frame, hog, bg, k):
    H, W = frame.shape[:2]

    # HOG detection
    sx = W / 480;  sy = H / 270
    small = cv2.resize(frame, (480, 270))
    boxes, weights = hog.detectMultiScale(small, winStride=(4, 4), padding=(8, 8), scale=1.05)

    hog_boxes = []
    for i, (x, y, bw, bh) in enumerate(boxes):
        conf = float(weights[i]) if i < len(weights) else 0
        if conf < 0.15:
            continue
        b = (int(x*sx), int(y*sy), int((x+bw)*sx), int((y+bh)*sy))
        if _valid_person(b, W, H):
            hog_boxes.append(b)

    # MOG2 background subtraction (supplementary)
    mask = bg.apply(small)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k, iterations=2)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  k, iterations=1)
    mask = cv2.dilate(mask, k, iterations=2)
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    MIN_A = (480 * 270) * 0.003
    MAX_A = (480 * 270) * 0.30
    PAD   = 8
    mog_boxes = []
```

```
    for c in cnts:
        a = cv2.contourArea(c)
        if a < MIN_A or a > MAX_A:
            continue
        x, y, bw, bh = cv2.boundingRect(c)
        if bh == 0 or (bh / max(bw, 1)) < 1.3:
            continue
        b = (max(0, int((x-PAD)*sx)), max(0, int((y-PAD)*sy)),
             min(W-1, int((x+bw+PAD)*sx)), min(H-1, int((y+bh+PAD)*sy)))
        if not _valid_person(b, W, H):
            continue
        if any(_iou(b, e) > 0.30 for e in hog_boxes):
            continue
        mog_boxes.append(b)

    all_boxes = _nms(hog_boxes + mog_boxes)
    return [{'bbox': list(b), 'landmarks': PoseKeypoints(b)} for b in all_boxes]


def _valid_person(b, W, H):
    x1, y1, x2, y2 = b
    bw = x2 - x1;  bh = y2 - y1
    if bw <= 0 or bh <= 0:
        return False
    asp = bh / bw
    if asp < 1.4 or asp > 5.0:
        return False
    h_ratio = bh / H
    if h_ratio < 0.08 or h_ratio > 0.90:
        return False
    if y2 / H < 0.40:
```

```
        return False
    return True


def draw_skeletons(frame, dets, color=(0, 220, 120)):
    for d in dets:
        kp = d.get('landmarks')
        if not kp:
            continue
        pts = kp.points
        for a, b in PoseKeypoints.CONNECTIONS:
            cv2.line(frame, (int(pts[a][0]), int(pts[a][1])),
                     (int(pts[b][0]), int(pts[b][1])), color, 2, cv2.LINE_AA)
        for pt in pts.values():
            cv2.circle(frame, (int(pt[0]), int(pt[1])), 3, color, -1, cv2.LINE_AA)
        nose = pts['nose'];  ls = pts['l_shoulder'];  rs = pts['r_shoulder']
        hr   = max(5, int(abs(rs[0] - ls[0]) * 0.22))
        cv2.circle(frame, (int(nose[0]), int(nose[1] - hr * 0.5)), hr, color, 2, cv2.LINE_AA)

def main():
    # Check video folder exists
    if not os.path.exists(VIDEO_DIR):
        print(f"[ERROR] Video folder not found: {os.path.abspath(VIDEO_DIR)}")
        print(f"        Create a folder called 'videos' and put your .mp4 files in it.")
        return

    videos = sorted([f for f in os.listdir(VIDEO_DIR) if f.endswith('.mp4')])
    if not videos:
        print(f"[ERROR] No .mp4 files found in '{VIDEO_DIR}' folder.")
        return
```

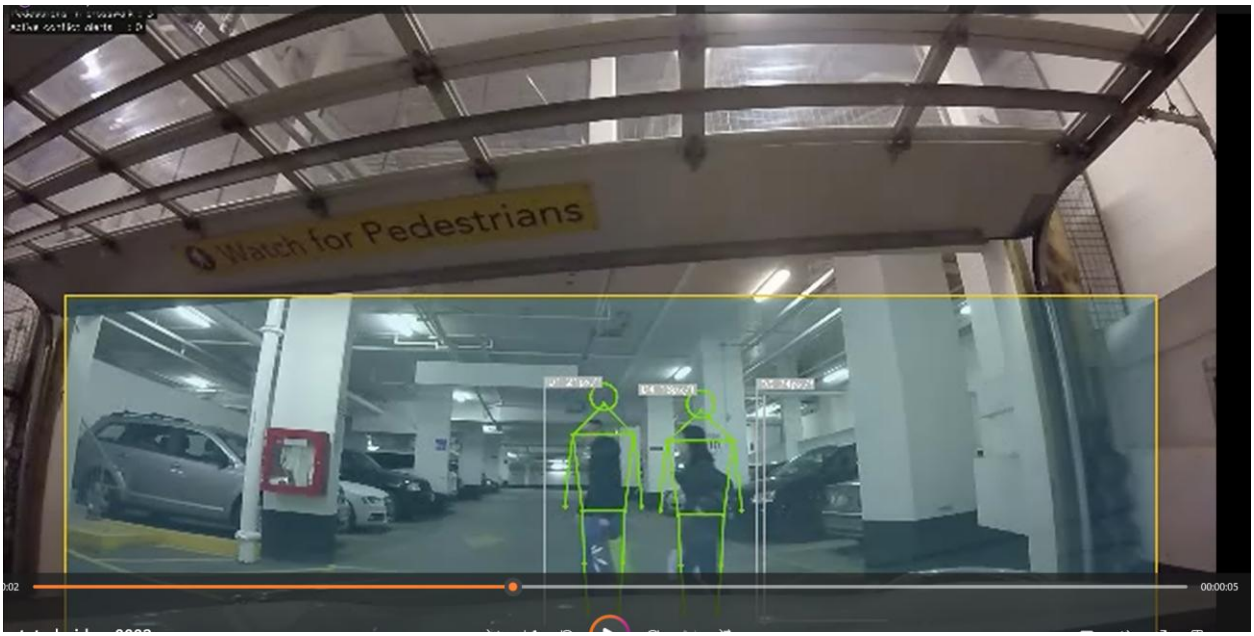
```
os.makedirs(OUTPUT_DIR, exist_ok=True)
os.makedirs(os.path.join(OUTPUT_DIR, "conflict_crops"), exist_ok=True)

print(f"\n{'='*60}")
print(f"  Pedestrian Conflict Detection  –  JAAD Dataset")
print(f"  Videos   : {len(videos)}")
print(f"  Video dir: {os.path.abspath(VIDEO_DIR)}")
print(f"  Output   : {os.path.abspath(OUTPUT_DIR)}")
print(f"  Pipeline : HOG + MOG2 + IoU Tracker + Rule Engine")
print(f"{'='*60}\n")

logger    = ConflictLogger(OUTPUT_DIR)
all_stats = []
grand_t0  = time.time()

for vi, vname in enumerate(videos, 1):
    vpath = os.path.join(VIDEO_DIR, vname)
    cap   = cv2.VideoCapture(vpath)

    if not cap.isOpened():
        print(f"[SKIP] Cannot open: {vpath}")
        continue

    W   = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    H   = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    FPS = cap.get(cv2.CAP_PROP_FPS) or 30.0
    TOT = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    poly = np.array([[int(x*W), int(y*H)] for x, y in DEFAULT_CROSSWALK_POLY],
                    dtype=np.int32)
```

```
    # Reset per video
    Track._id_counter = 0
    hog = cv2.HOGDescriptor()
    hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
    bg  = cv2.createBackgroundSubtractorMOG2(history=200, varThreshold=40,
detectShadows=False)
    k   = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    trk   = Tracker()
    rules = ConflictRuleEngine()

    ow, oh   = RESIZE_OUTPUT
    out_path = os.path.join(OUTPUT_DIR, f"annotated_{vname}")
    writer   = cv2.VideoWriter(out_path, cv2.VideoWriter_fourcc(*'mp4v'),
                        max(FPS / 2, 10), (ow, oh))

    stats = {'video': vname, 'frames': TOT, 'NC': 0, 'CE': 0, 'BL': 0, 'total': 0}
    fn = 0;  t0 = time.time();  seen = set()

    print(f"[{vi:02d}/{len(videos)}] {vname}  ({TOT} frames)", flush=True)

    while True:
        ret, frame = cap.read()
        if not ret:
            break
        fn += 1
        if fn % 2 != 0:
            continue

        dets       = detect_persons(frame, hog, bg, k)
        trk.update(dets)
        roi_tracks = [t for t in trk.tracks if foot_in_poly(t, poly)]
```

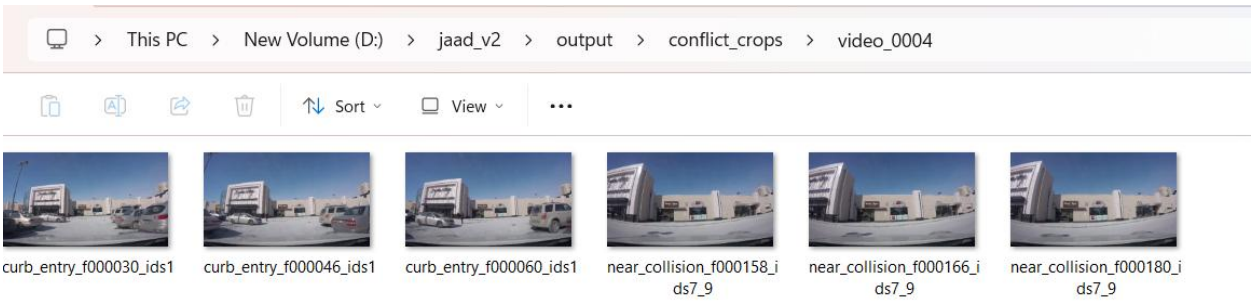
```
evs       = rules.update(trk.tracks, poly, frame.shape, fn)

for ev in evs:
    key = (vname, fn // 15, ev.conflict_type.name, tuple(sorted(ev.track_ids)))
    if key not in seen:
        seen.add(key)
        logger.log(ev, vname)
        skey = CTYPE_KEY.get(ev.conflict_type.name, 'BL')
        stats[skey]   += 1
        stats['total'] += 1
        if SAVE_CONFLICT_CROPS:
            cd = os.path.join(OUTPUT_DIR, "conflict_crops", vname[:-4])
            os.makedirs(cd, exist_ok=True)
            save_crop(cv2.resize(frame, (ow, oh)), ev, cd, fn)

# Draw
canvas = frame.copy()
draw_crosswalk_roi(canvas, poly)
draw_skeletons(canvas, dets)
draw_tracks(canvas, trk.tracks, crosswalk_poly=poly)
draw_conflicts(canvas, evs)
draw_hud(canvas, fn, fn / (time.time() - t0 + 1e-6),
         len(roi_tracks), len(evs), vname)
draw_legend(canvas)
writer.write(cv2.resize(canvas, (ow, oh)))

if fn % 60 == 0:
    pct = fn / max(TOT, 1) * 100
    print(f"    {pct:5.1f}%  frame {fn}/{TOT}"
          f"  conflicts={stats['total']}", flush=True)
```

```
        cap.release()
        writer.release()
        elapsed = time.time() - t0
        print(f"  Done in {elapsed:.0f}s  |  "
              f"NC={stats['NC']}  CE={stats['CE']}  BL={stats['BL']}  "
              f"total={stats['total']}\n")
        all_stats.append(stats)
    logger.close()

    print(f"\n{'='*62}")
    print(f"  {'Video':<24} {'Frames':>7} {'NC':>5} {'CE':>5} {'BL':>5} {'Total':>7}")
    print(f"  {'-'*57}")
    tot = {'frames': 0, 'NC': 0, 'CE': 0, 'BL': 0, 'total': 0}
    for s in all_stats:
        print(f"  {s['video']:<24} {s['frames']:>7} "
              f"{s['NC']:>5} {s['CE']:>5} {s['BL']:>5} {s['total']:>7}")
        for k2 in tot:
            tot[k2] += s[k2]
    print(f"  {'-'*57}")
    print(f"  {'TOTAL':<24} {tot['frames']:>7} "
          f"{tot['NC']:>5} {tot['CE']:>5} {tot['BL']:>5} {tot['total']:>7}")
    grand = time.time() - grand_t0
    print(f"\n  Total time : {grand:.0f}s")
    print(f"  Output dir : {os.path.abspath(OUTPUT_DIR)}")
    print(f"  CSV log    : {os.path.join(os.path.abspath(OUTPUT_DIR), 'all_conflicts.csv')}")
    print(f"  Videos     : {os.path.join(os.path.abspath(OUTPUT_DIR), 'annotated_*.mp4')}")
    print(f"{'='*62}\n")


if __name__ == "__main__":
    main()
```

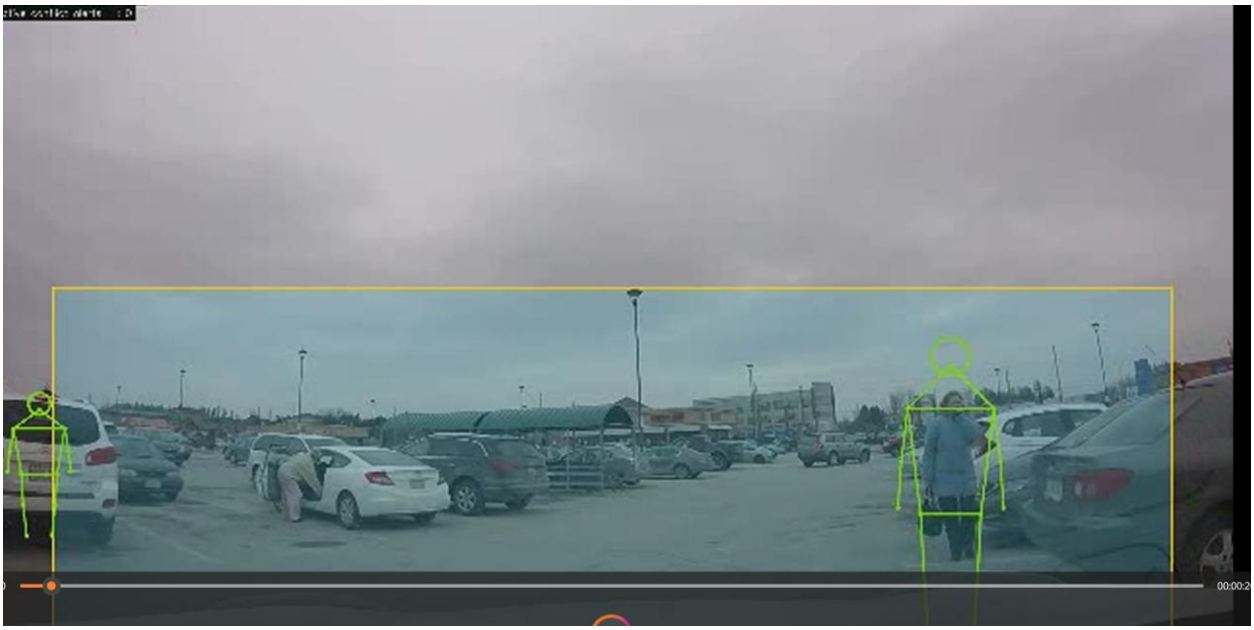
10.2 SCREENSHOTS



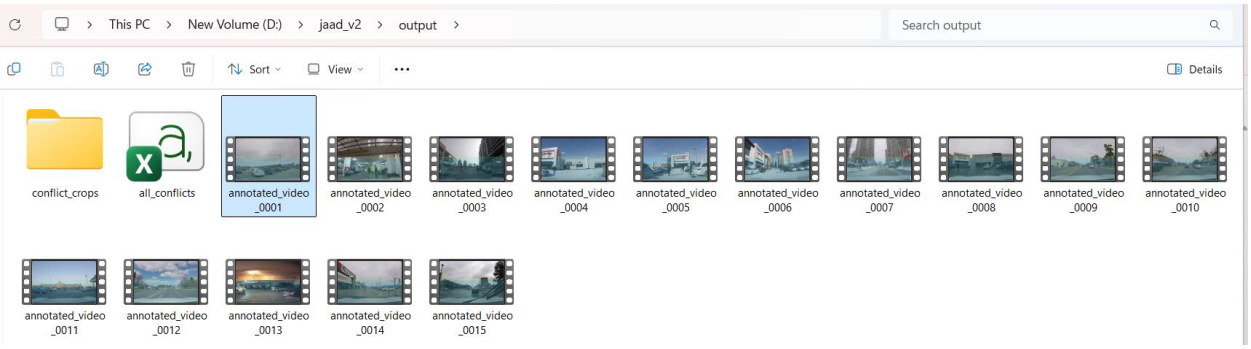
	A	B	C	D	E	F	G	H	I	J
1	video	timestamp	frame	conflict_ty	severity	track_ids	message			
2	video_000	2026-05-1	598	CURB_ENT	WARNING	[11]	Sudden curb entry ID 11 vx=34.0px/f			
3	video_000	2026-05-1	600	CURB_ENT	WARNING	[11]	Sudden curb entry ID 11 vx=34.0px/f			
4	video_000	2026-05-1	30	CURB_ENT	WARNING	[1]	Sudden curb entry ID 1 vx=110.0px/f			
5	video_000	2026-05-1	46	CURB_ENT	WARNING	[1]	Sudden curb entry ID 1 vx=110.0px/f			
6	video_000	2026-05-1	60	CURB_ENT	WARNING	[1]	Sudden curb entry ID 1 vx=110.0px/f			
7	video_000	2026-05-1	158	NEAR_COI	WARNING	[7, 9]	Near-collision IDs 7&9 dist=87px			
8	video_000	2026-05-1	166	NEAR_COI	WARNING	[7, 9]	Near-collision IDs 7&9 dist=88px			
9	video_000	2026-05-1	180	NEAR_COI	WARNING	[7, 9]	Near-collision IDs 7&9 dist=88px			
10	video_000	2026-05-1	34	CURB_ENT	WARNING	[1]	Sudden curb entry ID 1 vx=18.0px/f			
11	video_000	2026-05-1	46	CURB_ENT	WARNING	[1]	Sudden curb entry ID 1 vx=18.0px/f			
12	video_000	2026-05-1	60	CURB_ENT	WARNING	[1]	Sudden curb entry ID 1 vx=18.0px/f			
13	video_000	2026-05-1	28	CURB_ENT	WARNING	[9]	Sudden curb entry ID 9 vx=32.0px/f			
14	video_000	2026-05-1	30	NEAR_COI	CRITICAL	[7, 8]	Near-collision IDs 7&8 dist=30px			
15	video_000	2026-05-1	32	CURB_ENT	WARNING	[10]	Sudden curb entry ID 10 vx=170.0px/f			
16	video_000	2026-05-1	36	CURB_ENT	WARNING	[8]	Sudden curb entry ID 8 vx=28.0px/f			
17	video_000	2026-05-1	46	NEAR_COI	WARNING	[7, 8]	Near-collision IDs 7&8 dist=46px			
18	video_000	2026-05-1	46	CURB_ENT	WARNING	[10]	Sudden curb entry ID 10 vx=170.0px/f			
19	video_000	2026-05-1	50	CURB_ENT	WARNING	[1]	Sudden curb entry ID 1 vx=26.0px/f			
20	video_000	2026-05-1	60	CURB_ENT	WARNING	[1]	Sudden curb entry ID 1 vx=89.5px/f			
21	video_000	2026-05-1	60	CURB_ENT	WARNING	[10]	Sudden curb entry ID 10 vx=170.0px/f			
22	video_000	2026-05-1	54	CURB_ENT	WARNING	[3]	Sudden curb entry ID 3 vx=22.0px/f			
23	video_000	2026-05-1	16	CURB_ENT	WARNING	[1]	Sudden curb entry ID 1 vx=82.0px/f			
24	video_000	2026-05-1	30	CURB_ENT	WARNING	[1]	Sudden curb entry ID 1 vx=14.0px/f			
25	video_000	2026-05-1	100	CURB_ENT	WARNING	[6]	Sudden curb entry ID 6 vx=22.0px/f			




```
================================================================
  Video                        Frames    NC    CE    BL   Total
  ---------------------------------------------------------
  video_0001.mp4                  600     0     2     0       2
  video_0002.mp4                  210     0     0     0       0
  video_0003.mp4                  210     0     0     0       0
  video_0004.mp4                  180     3     3     0       6
  video_0005.mp4                  240     0     0     0       0
  video_0006.mp4                  330     0     3     0       3
  video_0007.mp4                  120     2     7     0       9
  video_0008.mp4                  150     0     1     0       1
  video_0009.mp4                  120     0     7     0       7
  video_0010.mp4                   90     0     0     0       0
  video_0011.mp4                  270     0     0     0       0
  video_0012.mp4                  120     0     0     0       0
  video_0013.mp4                  150     0     0     0       0
  video_0014.mp4                  270     8     0     0       8
  video_0015.mp4                  420     0     0     0       0
  ---------------------------------------------------------
  TOTAL                          3480    13    23     0      36

  Total time : 170s
  Output dir : D:\jaad_v2\output
  CSV log    : D:\jaad_v2\output\all_conflicts.csv
  Videos     : D:\jaad_v2\output\annotated_*.mp4
================================================================
```



Media Player
00:00:06
notated_video_0001


This PC
New Volume (D:)
jaad_v2
output
Search output
Sort
View
Details
conflict_crops
all_conflicts
annotated_video_0001
annotated_video_0002
annotated_video_0003
annotated_video_0004
annotated_video_0005
annotated_video_0006
annotated_video_0007
annotated_video_0008
annotated_video_0009
annotated_video_0010
annotated_video_0011
annotated_video_0012
annotated_video_0013
annotated_video_0014
annotated_video_0015